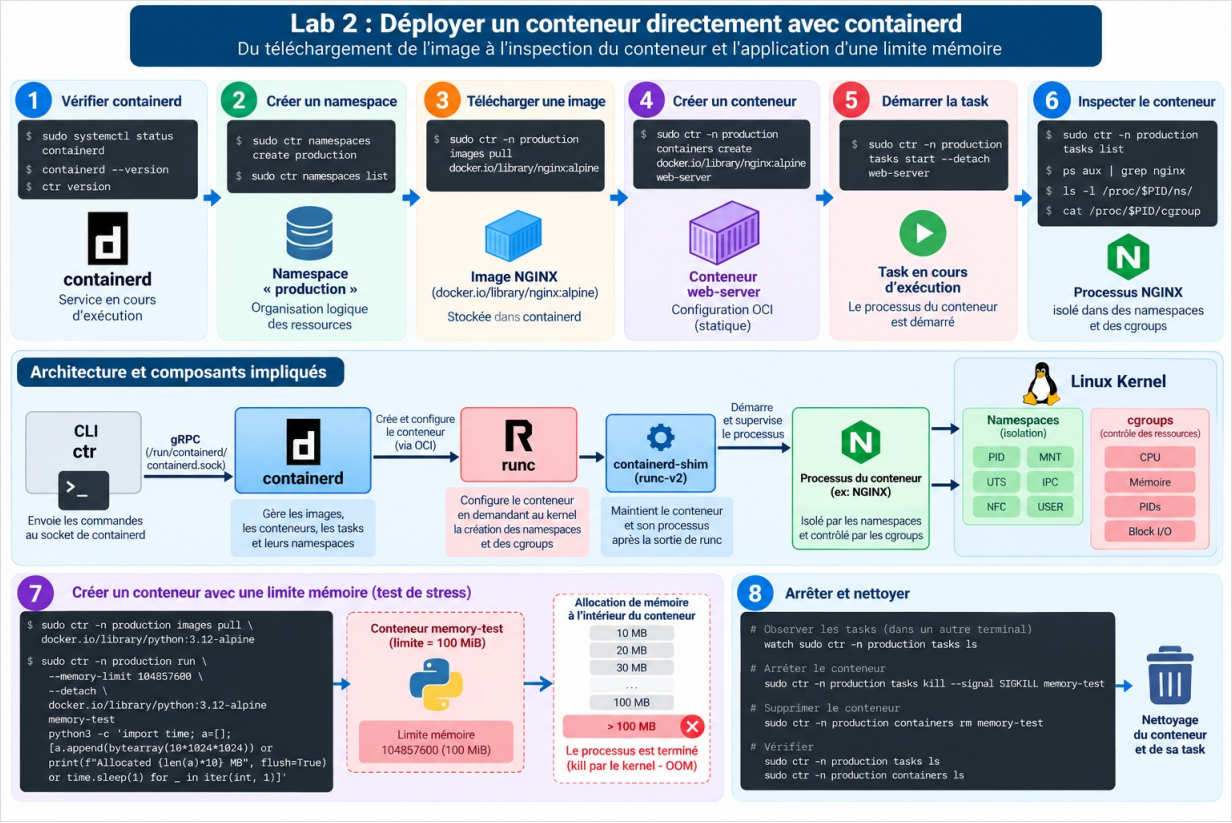
TP 2 Guidé: Déployer un conteneur directement avec containerd

https://docs.docker.com/engine/install/centos/

Vous êtes administrateur d'un serveur Linux utilisé comme nœud d'infrastructure.

Le serveur n'utilise **pas Docker**. Le moteur de conteneurs installé est **containerd**.



Votre mission consiste à :

- vérifier que `containerd` fonctionne ;
- utiliser directement son outil CLI `ctr` ;
- créer un espace de travail containerd ;
- télécharger une image ;
- créer un conteneur ;
- démarrer son processus ;
- observer les processus et les namespaces créés ;
- identifier le rôle de `containerd-shim` et de `runc` ;
- appliquer une limite de ressources ;

1. Vérifier l'environnement

Avant de commencer, vérifier que `containerd` est installé et fonctionne.

```
sudo systemctl status containerd
```

Vérifier la version :

```
containerd --version
```

Puis :

```
ctr version
```

Vérifier le socket Unix utilisé par containerd :

```
ls -l /run/containerd/containerd.sock
```

2. Créer un namespace containerd

containerd permet de séparer les ressources utilisées par différents consommateurs grâce à ses **namespaces**.

Nous allons créer un namespace appelé `production`.

```
sudo ctr namespaces create production
```

Vérifier :

```
sudo ctr namespaces list
```

> *Un namespace containerd est un mécanisme d'organisation/isolation logique des ressources de containerd. Ce n'est pas un namespace Linux et ce n'est pas une frontière de sécurité.*

3. Télécharger une image directement avec containerd

Notre serveur doit exécuter un serveur Web Nginx.

Télécharger directement l'image dans containerd :

```
sudo ctr -n production images pull docker.io/library/nginx:alpine
```

s'il ya un erreur Too many requests

```
Pull the Python Alpine image with Docker

 docker pull python:3.12-alpine

Export the image

 docker save python:3.12-alpine -o python.tar

Import into the production containerd namespace

 sudo ctr -n production images import python.tar

Verify

 sudo ctr -n production images ls | grep pythonar
```

Vérifier les images disponibles :

```
sudo ctr -n production images list
```

4. Créer un conteneur

Nous allons maintenant demander à containerd de créer un conteneur nommé `web-server`.

```
sudo ctr -n production containers create docker.io/library/nginx:alpine web-server
```

Vérifier les conteneurs :

```
sudo ctr -n production containers list
```

Vous devriez retrouver :

```
web-server
```

Container ≠ Task

À ce stade, le conteneur est **créé**, mais son processus n'est pas encore exécuté.

Nous avons donc :

```
Container
   |
   | configuration
   ▼
Task
   |
   | processus en exécution
   ▼
Process
```

Cette distinction est importante dans l'utilisation directe de containerd.

5. Démarrer la task

Démarrer maintenant le processus du conteneur :

```
sudo ctr -n production tasks start --detach web-server
```

Vérifier les tasks actives :

```
sudo ctr -n production tasks list
```

Le processus Nginx est maintenant réellement exécuté sur le système.

6. Observer le processus du conteneur

Depuis l'hôte, rechercher le processus Nginx :

```
ps aux | grep nginx
```

Vous pouvez également utiliser :

```
ps -ef | grep nginx
```

Identifier le PID du processus principal.

Par exemple :

```
PID=<PID>
```

Vérifier ensuite les informations du processus :

```
sudo cat /proc/$PID/status
```

7. Observer les Linux Namespaces

Dans le chapitre précédent, nous avons étudié les **Linux namespaces**.

Maintenant, nous allons vérifier qu'ils sont réellement utilisés par le conteneur créé par containerd.

Afficher les namespaces du processus :

```
sudo ls -l /proc/$PID/ns/
```

Observer individuellement quelques namespaces :

```
sudo readlink /proc/$PID/ns/pid
```

```
sudo readlink /proc/$PID/ns/mnt
```

```
sudo readlink /proc/$PID/ns/uts
```

```
sudo readlink /proc/$PID/ns/ipc
```

Selon la configuration du conteneur, vous pourrez observer des namespaces tels que :

```
pid
mnt
uts
ipc
net
cgroup
user
```

Le runtime OCI demande au kernel Linux de mettre en place les mécanismes nécessaires à l'isolation.

8. Observer le containerd-shim

Maintenant que le conteneur fonctionne, rechercher les processus liés à containerd :

```
ps -ef | grep containerd
```

Rechercher spécifiquement le shim :

```
ps -ef | grep containerd-shim
```

Selon la configuration utilisée, vous pourrez observer un processus similaire à :

```
containerd-shim-runc-v2
```

> *À retenir Le shim permet à containerd de maintenir la gestion du conteneur et de son processus sans que containerd soit directement le processus parent du programme exécuté dans le conteneur.*

9. Observer runc

Vérifier si un processus `runc` est actuellement visible :

```
ps -ef | grep runc
```

Il est possible que vous ne voyiez **pas** un processus `runc` permanent.

C'est normal.

`runc` est généralement utilisé pour créer/configurer le conteneur puis le processus du conteneur continue son exécution sous le contrôle de l'environnement créé.

10. Observer les cgroups du conteneur

Afficher le cgroup du processus :

```
sudo cat /proc/$PID/cgroup
```

Vérifier le système cgroup :

```
mount | grep cgroup
```

Puis :

```
ls /sys/fs/cgroup/
```

Selon la distribution et la configuration du système, vous pourrez observer une hiérarchie cgroup v2.

11. Créer un conteneur avec une limite mémoire

Nous allons maintenant créer un second conteneur avec une limite de mémoire.

```
sudo ctr -n production images pull docker.io/library/python:3.12-alpine
```

dans un autre terminal “watch” le conteneur

```
sudo ctr -n production tasks ls
```

puis retourne pour ajouter un conteneur de stress

```
sudo ctr -n production run \
  --memory-limit 104857600 \
  --detach \
  docker.io/library/python:3.12-alpine \
  memory-test \
  python3 -c 'import time; a=[]; print("Starting memory stress", flush=True);
[a.append(bytearray(10*1024*1024)) or print(f"Allocated {len(a)*10} MB", flush=True) or
time.sleep(1) for _ in iter(int, 1)]'
```

Le programme Python à l’intérieur du conteneur a progressivement alloué de la RAM :

```
10 MB → 20 MB → 30 MB → ... → 100 MB → dépassement
```

La commande :

```
--memory-limit 104857600
```

demande à containerd de configurer une limite mémoire pour le conteneur.

Vérifier ensuite l’état de conteneur dans l’autre terminal à l’aide du watch

Que se passe-t-il lorsque le conteneur atteint sa limite mémoire ? Le conteneur continue-t-il à fonctionner ? Quel composant du système Linux intervient lorsque la limite est dépassée ? Quelle différence observez-vous entre le container et le task après l’arrêt ? Pourquoi cette expérience montre-t-elle que le conteneur dépend des mécanismes du kernel Linux ?

> *Pour supprimer un conteneur containerd on doit tout d’abord killer le process puis le supprimer*

```
sudo ctr -n production tasks kill --signal SIGKILL containername
```

```
sudo ctr -n production tasks ls
```

```
sudo ctr -n production containers rm containername
```

```
sudo ctr -n production containers ls
```